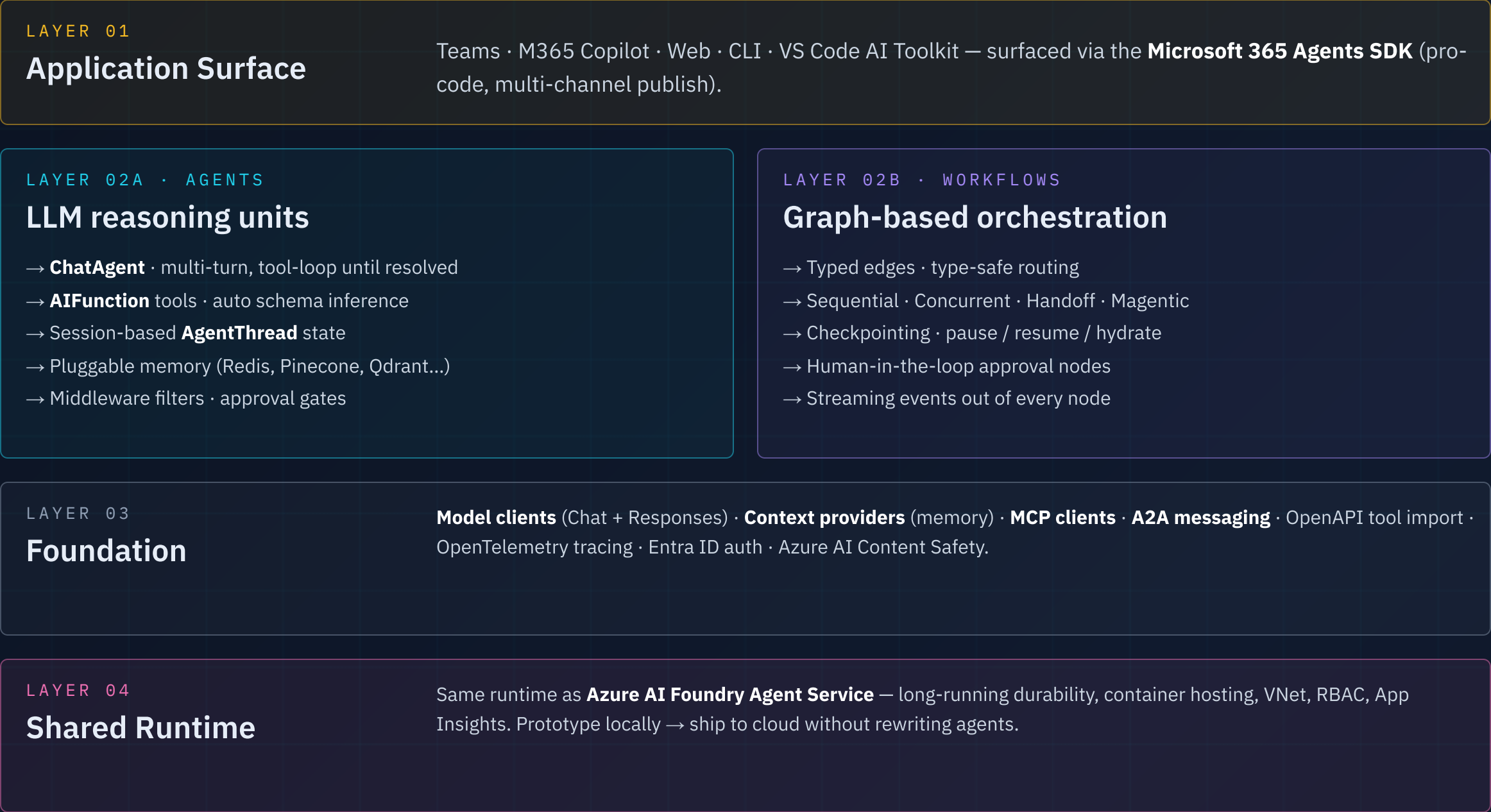


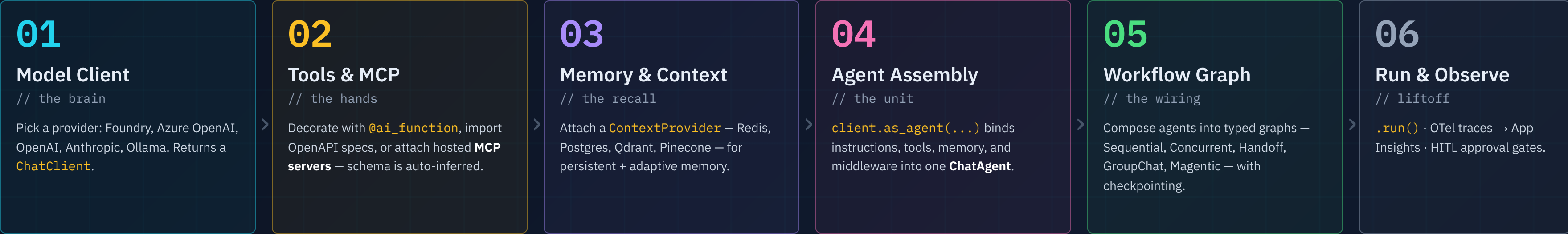
Deep Technical Architecture

The successor to Semantic Kernel + AutoGen — a unified, open-source SDK splitting **Agents** (intelligence) from **Workflows** (orchestration) on a shared runtime.



Pro-Code Agents • Wiring Flow

From `pip install` to a multi-agent workflow — six wiring stages where the SDK assembles model, tools, memory, and orchestration into a runnable agent graph.



```
// PYTHON • WIRING THE SIX STAGES

# 01 • Model client
from agent_framework.foundry import FoundryChatClient
from agent_framework import ai_function, ChatAgent, SequentialWorkflow

client = FoundryChatClient(project_endpoint="...", model="gpt-5.4-mini", credential=cred)

# 02 • Tool — schema auto-inferred from signature + docstring
@ai_function
def lookup_invoice(invoice_id: str) -> dict: return db.fetch(invoice_id)

# 03 + 04 • Memory + agent assembly
researcher = client.as_agent(name="Researcher", tools=[lookup_invoice], memory=redis_store)
reviewer   = client.as_agent(name="Reviewer",   instructions="Audit findings.")

# 05 • Workflow graph — typed, checkpointed, resumable
flow = SequentialWorkflow([researcher, reviewer]).with_checkpointing(store=blob)
```

// WHAT THE SDK GUARANTEES

What you get once it's wired

- ▶ **Tool-loop until resolved** — ChatAgent keeps calling tools until it has an answer.
- ▶ **Type-safe routing** — workflow edges validated at compile / load time.
- ▶ **Pause / resume / hydrate** — checkpoints survive process restarts.
- ▶ **Approval-gated tools** — emit pending request → UI → resume.
- ▶ **Same runtime as Foundry Agent Service** — no rewrite to ship.

Runtime & Orchestration Patterns

Five orchestration patterns ship in-the-box · one shared runtime carries the agent from a developer's laptop to **Azure AI Foundry** Agent Service without rewrite.

PATTERN 01

Sequential



Step-by-step pipeline. Agent N's output is Agent N+1's input. Best for deterministic doc-processing chains.

PATTERN 02

Concurrent



Fan-out, run in parallel, fan-in. Best for ensembling perspectives or independent sub-tasks.

PATTERN 03

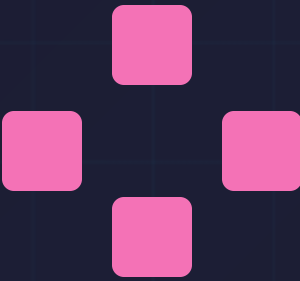
Handoff



Responsibility transfers between agents as context evolves — triage → specialist routing.

PATTERN 04

Group Chat



Agents brainstorm, debate, critique. Inherited from AutoGen — now durable + observable.

PATTERN 05

Magentic



Manager agent builds a dynamic task ledger, coordinates specialists (and humans) for open-ended problems.

// THE RUNTIME JOURNEY

One agent definition, three environments

ENV 01 · LOCAL

Developer Inner Loop

- **VS Code AI Toolkit** — create, run, visualize multi-agent flows
- In-process runtime · breakpoints across agent boundaries
- OTel traces stream to a local collector

ENV 02 · CONTAINER

Self-Hosted / Multi-Cloud

- Cloud-agnostic runtime · Docker / K8s / on-prem
- Bring your own model provider (OpenAI, Anthropic, Ollama...)
- OTel → any APM backend

ENV 03 · MANAGED

Azure AI Foundry Agent Service

- **Shared runtime** — same agent code, no rewrite
- VNet · Entra ID · RBAC · Content Safety
- App Insights · long-running durability · hydration

KEY TAKEAWAY · The pro-code wiring you author once survives every environment — local debug, container, and Foundry — because the runtime, telemetry contract, and orchestration primitives are identical at every layer.